Hudbot / Cavebot — Comprehensive Technical Analysis

Read-only exploration. No files in the cloned repo were modified.

Sources analyzed
- Repo: `https://github.com/TheNabu222/hudbot` → cloned to `/home/ubuntu/github_repos/hudbot`
- Main builder: `App.tsx` (28,293 lines / 1.4 MB), `types.ts`, `utils/*`, `components/*`
- Abacus pure-HTML version: `peripheral/alt_version(abacus_pure_html)/`
- Game data export: `/home/ubuntu/Uploads/7262026_stripped.json` (the "cavebot" project)
- Scene reference: `/home/ubuntu/Uploads/IMG_6080 2.pdf` (80 gameplay screenshots)

---

1. Executive Summary

Hudbot is a visual, no-code, drag-and-drop builder for point-and-click adventure / life-sim games that exports to a standalone HTML/JS runtime (targeting Neocities). The project the user is building with it is **"cavebot"** — a prehistoric ("Paleo") narrative life-sim starring a cave-dweller, an extended settlement of NPCs (Gilgrokmesh, Nin, En, Lunamkita, the children, etc.), animal companions (a hyena, a wolf pup), and a central **shoebill courtship** arc that culminates in a surreal "MechAnzu / Backrooms" reality-glitch transformation.

There are **two implementations** in the repo:

	Main builder	Abacus pure-HTML version
Location	repo root (`App.tsx`)	`peripheral/alt_version(abacus_pure_html)/`
Stack	React 19 + TypeScript + Vite	Zero-dependency vanilla HTML/CSS/JS
Codename	"Nabu / Neocities Game Builder"	"Anzu Game Studio" (Phase 1–3)
Size	One 28k-line monolith + utils	~30 small modular files (~10.8k lines total)
Maturity	Feature-rich but fragmented	Clean architecture, fewer features (foundational)
Runtime	`utils/exportHtml.ts` (3,617 lines) re-implements logic	Modular `js/engine/*` registry inlined into export

The uploaded `7262026_stripped.json` is a real, substantial project export: **31 scenes, 208 assets, 36 characters, 10 dialogue trees, 37 items, 9 quests, 5 factions, 3 maps**. It is a "stripped" ex-

port — image/audio binary `dataURL`s have been removed to shrink the file (159 of 208 assets have an empty `src`).

The single most important structural problem, confirmed by the repo's own `NABU_GAME_BUILDER_AUDIT.md` and by direct inspection of the JSON, is **state fragmentation**: game logic is expressed through dozens of loosely-related, free-text "flags," duplicated across `App.tsx` and `exportHtml.ts`, with no unified rule/event system. This makes complex arcs (the shoebill courtship, multi-stage quests, scene anomalies) brittle and hard to author.

---

2. Repository Structure

```
hudbot/
├── App.tsx                      # 28,293-line React monolith (the entire editor UI +
editor-side runtime)
├── index.tsx / index.html       # Vite entry
├── types.ts                     # 743 lines — the canonical data model (Project + all
sub-types)
├── components/                  # 16 extracted React components (editors, pickers,
modals)
│   ├── MapMaker.tsx, RuleConditionEditor.tsx, ClickResponseEditor.tsx,
│   ├── AssetLibraryManager.tsx, ImageEditorModal.tsx, StudioWorkflowNav.tsx, ...
├── utils/
│   ├── exportHtml.ts            # 3,617 lines — generates the standalone playable HTML
(its own runtime)
│   ├── runtimeRules.ts          # condition evaluation (flag/item/quest/skill/need/re-
lationship/time)
│   ├── questObjectives.ts, crafting.ts, projectPersistence.ts,
│   ├── cavebotTemplate.ts       # 1,819 lines — a starter "cavebot" template project
│   ├── twineAdapter.ts, templates.ts, html.ts, fileHelpers.ts
├── templates/
│   ├── JSON_EXPORT_FORMAT.md    # (aspirational/partial modular export format — see §7
caveat)
│   └── HTML_EXPORT_FORMAT.md
├── scripts/                     # smoke-export.mjs, smoke-project-compat.mjs (parity
smoke tests)
├── FEATURE_INVENTORY.md         # feature catalogue
├── NABU_GAME_BUILDER_AUDIT.md   # THE key document: capability matrix + fragmentation
audit + roadmap
├── assets/                      # 5,986 GIF + 2,824 PNG + 711 WAV + ... (large media
library)
└── peripheral/
    ├── alt_version(abacus_pure_html)/   # the clean vanilla-JS "Anzu" studio (ana-
lyzed in §5)
    ├── coaiexist-studio-package/
    └── scene/entropic-scene-maker-98/   # a separate scene-maker experiment
```

Total tracked files: **10,163** (overwhelmingly binary game assets). The actual application logic lives in `App.tsx`, `types.ts`, `utils/`, `components/`, and the `peripheral/alt_version(abacus_pure_html)/` tree.

---

3. The JSON Data Schema — Field-by-Field Breakdown

The root object is a `Project` (defined in `types.ts`). Verified against `7262026_stripped.json`.

3.1 Root `Project`

Key	Type	Notes (observed values in cavebot export)
`id`	string	`"21b6ac3c-…"`
`name`	string	`"My Neocities Game"`
`currentSceneId`	string	active scene id
`currentUiMenuId`	string | null	active UI menu overlay
`assets`	`Asset[]`	208
`globalSettings`	object	55 keys (see §3.11)
`scenes`	`Scene[]`	31
`uiMenus`	`Scene[]`	2 — same shape as Scene, used as overlays
`dialogueTrees`	`DialogueTree[]`	10
`inventoryItems`	`InventoryItem[]`	37
`craftingRecipes`	`CraftingRecipe[]`	2
`quests`	`Quest[]`	9
`maps`	`FastTravelMap[]`	3
`gameFlags`	`string[]`	15 — **free-text sentences used as flag identifiers**
`loreEntries`	`LoreEntry[]`	6
`factions`	`Faction[]`	5
`companions`	`Companion[]`	2
`characters`	`Character[]`	36
`prefabs`	`SceneObject[]`	1 — reusable object templates
`assetCategories`	`string[]`	2

3.2 `Asset`

```
id: string                // e.g. "github:assets/_cavebot-assets/....png" or a uuid
type: "image"|"hitbox"|"audio"|"script"|"video"|"ui_element"|"text"
category: string          // "edited/images", "root", ...
src: string               // URL, data-URL, or "" if stripped
dataURL?: string          // base64 payload (stripped out in this export)
name, width?, height?
exportSource?: "linked"|"github_inferred"|"embedded_fallback"
exportReason?: string     // human explanation of how it will export
isFavorite?, tags?[], description?, lore?, needsAttention?
trimStart?, trimEnd?, volume?   // audio-only
```

Observed: 199 image + 9 audio. `exportSource` breakdown: 159 `embedded_fallback` (empty src, baked data was stripped), 39 `linked`, 10 `github_inferred`. GitHub-id assets resolve to `https://raw.githubusercontent.com/thenabu222/entropic-ai/main/<path>`.

3.3 `Scene` (and `uiMenus`, same shape)

```
id, name, width:int, height:int, backgroundColor:string
objects: SceneObject[]
bgmAssetId?: string
// UI-menu-only flags:
isOpenByDefault?, blocksClicks?, closeOnClickOutside?: bool
```

3.4 `SceneObject` — the heaviest type (60+ optional fields)

Core transform + identity:

```
id, name, src, _assetId?
x, y, width, height, rotation, zIndex, opacity, locked
flipX?, flipY?, hidden?
cursor: CursorType, cursorAssetId?
```

Behavior / interaction:

```
interaction: InteractionType          // see enum below
interactionData?: string              // target id / payload (scene id, ui id, flag,
number...)
clickResponses?: ClickResponse[]      // ORDERED multi-action stack
conditionMode?: "all"|"any"; conditions?: RuleCondition[]
triggerOnEnter?, triggerOnce?, ignoreClicks?
giveItemId?, dialogueTreeId?, scriptAssetId?
requireItemId?, consumeRequiredItem?
showIfFlag?, hideIfFlag?              // legacy single-flag visibility gating
```

Relationship / RPG:

```
characterId?, affinityId?, parentObjectId?
requiredSkill?, skillCheckDifficulty?, grantSkill?, grantSkillValue?
timeCost?, needsEffect?: Record<string,number>
reputationEffect?: { npcId, value }   // NOTE inconsistent target model (see §6)
flavorText?
```

Visual/animation/physics:

```
animation: AnimationType, animationDuration?, animationEasing?
blendMode: BlendMode, parallaxSpeed:number, filters?{bright-
ness,contrast,saturate,hueRotate,blur,sepia,invert,grayscale}
hasPhysics, physicsStatic?, physicsBounciness?, physicsFriction?, physicsDensity?
audioSrc?
```

Type discriminators + text tool:

```
isHitbox?, isScript?, isVideo?, isText?, isUiElement?, isDraggable?
textContent?, textColor?, textFontSize?, textFontFamily?, textAlign?, textStyle?,
textOutline?, textLetterSpacing?, textLineHeight?, textShadow?, textWeight?
```

UI-maker fields (when `isUiElement`):

```
uiElementType: "panel"|"button"|"progress"|"toggle"|"icon"|"tooltip"|"selection"|
               "image"|"text"|"inventory_grid"|"journal_text"|"quest_list"|"stat_list"
uiColorPrimary?, uiColorSecondary?, uiIconType?, uiValue?, uiChecked?, uiBorderType?,
uiBorderRadius?, uiBindingType?:
"none"|"need"|"flag"|"inventory_count"|"inventory"|"journal"|"quests"|"stats"
uiBindingId?, uiGridColumns?, uiGridRows?, uiGridGap?, uiPadding?, uiEmptyText?,
uiTextSource?, uiAnchor?
```

Responsive positioning:

```
stretchToScreen?, pinToEdge?, objectFit?, customCssClasses?
```

Across 31 scenes there are **334 scene objects**. Interaction distribution: `none` =297, `scene_change` =14, `dialogue` =11, `toggle_flag` =4, `collect` =2, then singletons of `start_quest`, `open_crafting`, `open_ui`, `sound`, `set_flag`, and **`start-dialogue`** (a hyphenated legacy value not in the current enum — see §6).

3.5 `InteractionType` enum (from `types.ts`)

`none, dialogue, scene_change, link, sound, skill_check, give-item, collect, run_script, save_game, load_game, toggle_inventory, toggle_needs_hud, toggle_skills_hud, open_ui, close_ui, modify_number, open_crafting, open_quest_log, start_quest, complete_quest, complete_quest_objective, open_skills, open_almanac, unlock_lore_entry, show_lore_entry, open_map, open_relationships, open_settings, set_flag, clear_flag, toggle_flag, show_object, hide_object, toggle_object, play_cutscene, restart_scene, restart_game, toggle_fullscreen, toggle_mute, advance_day, gift_item, exit_game`

3.6 `ClickResponse` (ordered action stack on an object)

```
id, interaction: InteractionType, interactionData?
giveItemId?, dialogueTreeId?, targetUiId?, scriptAssetId?
conditionMode?: "all"|"any"; conditions?: RuleCondition[]
triggerOnce?
```

24 click-responses exist across the project. Per the audit, secondary responses cannot currently carry the full set of conditions/RPG effects that the primary interaction can.

3.7 `RuleCondition` (the shared condition primitive)

```
id
type:
"flag"|"item"|"quest_active"|"quest_completed"|"skill"|"need"|"relationship"|"time"
targetId: string
comparator: "is"|"is_not"|"at_least"|"at_most"|"greater_than"|"less_than"
value?: number | boolean
```

Evaluated by `utils/runtimeRules.ts::evaluateRuleCondition` against `RuntimeGameState`.

3.8 Dialogue: `DialogueTree` → `DialogueNode` → `DialogueChoice`

```
DialogueTree { id, name, nodes: DialogueNode[], startNodeId: string|null }
DialogueNode { id, speaker, text, choices: DialogueChoice[], speakerAssetId?, por-
traitPosition?:"left"|"right" }
DialogueChoice {
  id, text, nextNodeId: string|null,
  requiredGameFlag?, setGameFlag?,
  startQuestId?, completeQuestId?, completeQuestObjectiveId?,
  unlockLoreEntryId?, showLoreEntryId?,
  giveItemId?, consumeItemId?, playSoundAssetId?, changeSceneId?,
  requiredSkillId?, requiredSkillValue?, grantSkillId?, grantSkillAmount?,
  timeCost?, needsEffect?: Record<string,number>,
  reputationEffect?: { factionId?, characterId?, value }
}
```

Observed: 10 trees, 45 nodes. One tree (“New Conversation”) has `startNodeId: null` (orphan/empty). Choice fields actually used in cavebot: `reputationEffect, setGameFlag, startQuestId, giveItemId, changeSceneId, grantSkillId, requiredGameFlag, completeQuestId`.

3.9 Items & Crafting

```
InventoryItem {
  id, name, description, iconAssetId: string|null,
  category?: "normal"|"consumable"|"ingredient"|"quest"|"crafting_station",
  collectionCategory?, isUsable?, consumeOnUse?, useMessage?, useSoundAssetId?, cus-
tomUseScript?,
  statRestores?: {stat, amount}[], combinations?: ItemCombination[]
}
ItemCombination { withItemId, resultItemId: string|null, destroyTarget, destroySelf,
successMessage? }
CraftingRecipe {
  id, name, ingredient1Id, ingredient2Id, ingredient3Id?, resultItemId,
  destroyIngredient1/2/3?, successMessage,
  requirements?: CraftingRequirement[], outcomes?: CraftingOutcome[]   // newer,
richer model
}
```

Observed item categories: ingredient×12, quest×6, consumable×4, crafting_station×4, normal×1, and **10 items with no `category`** (undefined). Note the **two parallel crafting models**: fixed `ingredient1/2/3Id` slots (legacy) vs. the flexible `requirements[]` / `outcomes[]` arrays (newer). Inventory is ID-based with **no quantities/stacks**.

3.10 Quests, Maps, Lore, Characters, Factions, Companions

```
Quest { id, name, description, objectives: QuestObjective[], rewards: QuestReward[],
autoStart? }
QuestObjective { id,
type:"talk_to"|"collect_item"|"reach_scene"|"skill_check"|"custom_flag",
                 targetId, description, requiredAmount? }
QuestReward   { type:"give_item"|"modify_status"|"set_flag", targetId, amount? }

FastTravelMap { id, name, backgroundSrc:string|null, backgroundFit?, backgroundScale?,
                backgroundOffsetX/Y?, nodes: MapNode[] }
MapNode { id, name, x, y, targetSceneId:string|null, iconSrc?, unlockedByDefault?, re-
quiredFlagId? }

LoreEntry { id, title, content, category?, entryType?:"lore"|"journal"|"quest_note",
questId?, requiredFlagId? }

Character {
  id, name, portraitAssetId?, factionId?, description?,
  relationshipTrackName?, defaultAffinity?,
  thresholds?: RelationshipThreshold[],   // {value,label,color}
  giftPreferences?: GiftPreference[],      // {itemId, change, reactionText?}
  relationships?: CharacterRelationship[]  // {characterId, kind, label, value, isMu-
tual?, isSecret?, notes?}
}
Faction { id, name, description, defaultAffinity, role?, reputationLabel?, joinFla-
gId?, allyFactionId?, rivalFactionId? }
Companion { id, name, characterId?, assetId:string|null, dialogueTreeId:string|null,
requiredFlagId?, interjections?: string[] }
```

Observed: 36 characters (Penzer, Henbur, Garza, Namluh, Lunamkita, Gilgrokmesh, Amaedina, Ninĝirila, Nin, En, Urgalkua, Inimdub, …), each with a full 7-tier threshold ladder (Hostile → Unfriendly → Wary → Neutral → … up to bonded/beloved). Factions: The Children, Trades Guild, Hyenas, Elders, Chieftain's Family. 9 quests with mixed objective types.

3.11 `globalSettings` (55 keys)

Groups: day/night (`useDayNightCycle` , `dayNightStartHour` , `dayNightHoursPerTick` , `dayNightTickMs`), system toggles (`enableNeeds` , `enableTTRPGStats`), stage (`stageWidth` =800, `stageHeight` =600, `snapToGrid` , `gridSize`), dialogue layout (`dialoguePosition` ="below", width/height/text/portrait sizing, `typewriterSpeed`), a large HUD block (`hideAllDefaultHud` , per-button hide flags, per-HUD enable flags, per-HUD scale/position/scaleX/scaleY), theming (`uiTheme` ="retro", `uiColorPrimary/Secondary/Background` , `uiBorderRadius` , `uiFontFamily` ="'Press Start 2P', monospace", `customCss`), `deviceFrame` (bezel + screen rect + `controls[]`), `hudOverlay` , `customCursorAssetId` , and custom stat definitions: `customSkills[]` (Cooking, Hafting, …), `customNeeds[]` (Meter 1–5), `customSkillDefinitions` / `customNeedDefinitions` (`StatTrackDefinition` maps), and `itemGroups[]` .

---

4. What the Existing Hudbot Builder Does

4.1 Authoring workflow (from `StudioWorkflowNav.tsx`)

Top-level studios: **Asset Library → Scene Studio (Rooms) → Interface Studio (Screen UI) → Conversations → Quests → Roster → Almanac → World Map.** The Scene Studio itself has a 4-step sub-flow: **Collect → Compose → Behaviors → Connect** (gather assets, place/transform them, wire click behavior, link scenes together).

4.2 Feature set (verified in `App.tsx` / `components/` / `utils/`)

- **Multi-scene canvas editor** — 800×600 stage, drag/resize/rotate, z-order layers, snap-to-grid, ghost outlines, flip, opacity, blend modes (16), CSS filters, parallax, per-object cursors (incl. custom cursor assets), physics (matter.js).
- **Rich object types** — images/GIFs, hitboxes, text objects (with narrative/speech/thought/sign styles), video cutscene objects, script objects, and **UI-maker elements** (panels, buttons, progress bars, toggles, inventory grids, quest lists, stat lists) with data-binding (`uiBindingType` / `uiBindingId`).
- **Asset library** — upload, tag, favorite, categorize, image editor/cropper (`ImageEditorModal`), GitHub-linked assets, audio metadata (trim/volume). Assets can export linked, github-inferred, or embedded-fallback.
- **Branching dialogue** — trees/nodes/choices with portraits, per-choice conditions (`requiredGameFlag`, `requiredSkillId/Value`) and consequences (set flag, give/consume item, change scene, start/complete quest, grant skill, needs effect, reputation effect, unlock lore).
- **Interaction system** — a per-object `interaction` plus an ordered `clickResponses[]` stack, each gated by `RuleCondition[]` with `all` / `any` mode; visibility gating via `showIfFlag` / `hideIfFlag`.
- **RPG systems** — inventory + item use/combination, three-slot + flexible crafting, quests + objectives + rewards, custom **skills** and **needs** (decay + HUD), TTRPG skill checks, day/night clock, factions + per-character **relationship tracks with thresholds, gift preferences, and a social graph**, companions with interjections, lore/journal/quest-note almanac, fast-travel maps with flag-gated nodes.
- **Presentation** — themes, custom fonts (default pixel `Press Start 2P`), device-frame bezel with clickable frame controls, HUD overlay image, configurable dialogue box.
- **How scenes connect** — three mechanisms: (1) object `interaction:"scene_change"` with `interactionData` = target scene id; (2) dialogue choice `changeSceneId`; (3) `FastTravelMap` nodes → `targetSceneId` (optionally gated by `requiredFlagId`).
- **Export** — `utils/exportHtml.ts` bakes the whole project into a single standalone HTML file with an embedded vanilla-JS runtime (Neocities-ready). `utils/twineAdapter.ts` offers Twine import/export. Persistence via `utils/projectPersistence.ts` (idb-keyval + JSON import/export). Smoke tests in `scripts/`.

4.3 The dialogue/scene look (from `IMG_6080 2.pdf`, 80 in-game screenshots)

The screenshots are pure **gameplay** (no editor UI). They confirm the intended feel:
- **Illustrated point-and-click rooms** in a mixed pixel-art + painterly style: sunset wetlands with a shoebill and foragable flowers, cozy firelit cave interiors (with hyena companion, cave paintings, spears, an abacus), forest shelters, and a beach/cove.
- **Needs HUD** top-right: green segmented bars labelled Hunger, Connection, Spiritual, Novelty (the `customNeeds`).
- **Dialogue system**: bottom overlay box (black, white monospace `Press Start 2P` text) matching `dialoguePosition:"below"`, with a **speaker portrait in the top-right corner** (e.g., the protagonist's stern portrait, Gilgrokmesh speaking "…The Matrix was crashing because it could not handle my raw charisma…"). Both narrative lines and `Speaker: "…"` lines appear.
- **Cast/companions on stage**: Gilgrokmesh (muscular caveman), the children (Gizzal, Garza), Nin/En, a wolf pup, the shoebill.
- **Story-phase / anomaly scenes**: surreal "glitching" imagery — a floating eye, a star-filled void portal, crystalline lightning trees over a ziggurat, a black-hole sun, MechAnzu/spirit-bird courtship visu-

als with floating hearts — i.e., the Paleo → Glitching → Backrooms progression the audit describes, currently achieved by hand-built duplicate scenes.

---

5. What the Abacus Pure-HTML Version Does Differently / Better

Location: `peripheral/alt_version(abacus_pure_html)/` — branded **“Anzu Game Studio,” “Phase 1: Core Foundation + Scene Builder.”** Zero dependencies; open `index.html` directly.

5.1 Architecture (the key contrast)

Where the main app is one 28k-line `App.tsx` with a separate 3.6k-line export runtime, the Anzu version is **~30 small, single-responsibility modules** loaded in order from `index.html`:

```
utils, state, toast, assets, canvas, layers, properties, hitbox, clickbox-inspector,
scenes, project, preview, export, shortcuts, ai-analysis, asset-tools, asset-manager,
github-bridge, rpg-* (needs/reputation/quests/dice/daynight/status/npc/systems),
game-flags, dialogue, inventory, save-load, transitions,
engine/* (runtime-core, dialogue, inventory, needs, reputation, quests, stats, time,
          status, npc, save-load),  engine-loader, engine-features-panel,
theme-system, help-system, settings-panel, app
```

5.2 Three things it does genuinely better

1. **A modular, toggleable runtime “engine” registry.** Each runtime feature lives in `js/engine/*.js` and registers itself into `window.EngineModuleRegistry` with `{id, label, featureKey, description, source}`. `engine-loader.js` keeps an ordered list (`core, dialogue, inventory, needs, reputation, quests, stats, time, status, npc, saveload`) and `generateInlineScripts()` inlines **only the enabled modules** into the exported HTML. Games ship with exactly the systems they use, and — crucially — **the exact same module source runs in editor Preview and in export**, which directly attacks the “editor/export divergence” problem the main app suffers from. (Caveat: in this Phase-1–3 snapshot the engine modules are still thin scaffolding/stubs; the heavy editor logic still lives in the top-level `js/*.js` files.)
2. **A defensive import/compatibility layer: `ProjectCompatibility.normalize()` (in `state.js`).** It ingests a foreign/legacy project and repairs it into a consistent internal shape — deriving `stageWidth/Height` from multiple possible sources, resolving asset sources (`dataURL || src || github: → raw.githubusercontent URL`), synthesizing missing asset records from object `src`, backfilling ids/defaults, and **mapping the enum drift** (`interaction → clickAction`, `backgroundColor → bgColor`, `_assetId → assetId`, `currentSceneId → activeSceneId`, `dialogue → start-dialogue`). This is exactly the kind of normalization the main app lacks and the reason legacy values like `start-dialogue` still appear in exports.
3. **Simplicity & inspectability.** Clean separation of concerns (state + undo/redo isolated in `state.js`, a dedicated `clickbox-inspector`, `github-bridge`, `theme-system`, `help-system`), a documented keyboard-shortcut scheme, and 30 readable files instead of one monolith — far easier to reason about, test, and extend than `App.tsx`.

5.3 Where it is weaker

It is explicitly a foundational build (README’s “Future Phases” still lists inventory, NPCs, save/load, conditional logic as upcoming). The main React app is far ahead on breadth: UI-maker, device frames,

gift/threshold relationship system, crafting, maps, lore almanac, image editor, Twine adapter. The ideal is the **Anzu architecture carrying the main app's feature set.**

---

6. Fragmentation & Continuity Issues (evidence-based)

These combine the repo's own `NABU_GAME_BUILDER_AUDIT.md` findings with concrete defects found in `7262026_stripped.json`.

6.1 Confirmed data defects in the cavebot export

- **Free-text sentences as flag IDs.** All 15 `gameFlags` are full sentences (`"Acquired Shoddy Spear...much to my chagrin."`, `'It seems the Shoebill is a fan of Gilgrokmesh\'s "merch..."'`). Flags are matched by exact string, so editing the display wording silently breaks every reference — a fragile, un-refactorable identity model.
- **Empty-string flag target.** Quest "Who Let the Fox Out?" has a `custom_flag` objective with `targetId: ""` — an objective that can never be satisfied/identified.
- **Quest objectives overload `targetId`.** `custom_flag` objectives store narrative prose in `targetId`; `reach_scene` / `collect_item` store real ids. No type-safety — the same field means very different things.
- **1 broken scene link.** Scene lunamkita has an object "Portal / Door" with `interaction:"scene_change"` but `interactionData: null` → dead exit. (13 valid, 1 broken.)
- **1 broken dialogue scene link.** A dialogue choice sets `changeSceneId: "8ef0b570-…"` which is not a scene that exists.
- **1 broken item reference.** A dialogue choice `giveItemId: "432442dc-…"` points to a non-existent item.
- **Duplicate scene names.** Two scenes are both named `"meethyenaba"` (plus a `"Start Scene (Copy)"`), an artifact of the copy-scene-to-simulate-variants pattern.
- **Legacy enum value in live data.** An object uses `interaction:"start-dialogue"` (hyphenated) — not a member of the current `InteractionType` union (`dialogue`), i.e., schema drift that only survives because the runtime is lenient.
- **Orphan dialogue tree.** "New Conversation" has `startNodeId: null`.
- **Uncategorized items.** 10 of 37 items have no `category`.
- **Stripped assets.** 159/208 assets have empty `src` (baked data removed). The runtime relies on `exportSource` / `exportReason` heuristics and GitHub URL inference to re-resolve them — fragile if the repo path or filenames move.

6.2 Systemic fragmentation (from the audit + code)

- **Flag explosion / no unified logic view.** The shoebill courtship alone needs ~13 hand-coordinated flags (`shoebill_met`, `shoebill_berries_given`, `flute_repair_1/2`, `shoebill_song_ready`, `mechanzu_triggered`, …). Nothing shows how flags relate, what sets them, or what they unlock.
- **Inconsistent relationship target model.** Reputation is referenced by `npcId` in `SceneObject.reputationEffect` but by `factionId` / `characterId` in `DialogueChoice.reputationEffect`, and `affinityId` / `characterId` on objects — three overlapping ways to name the same target.
- **Two crafting models coexist** (fixed `ingredient1/2/3Id` vs. `requirements[]` / `outcomes[]`); two flag/visibility models coexist (`showIfFlag` / `hideIfFlag` vs. `RuleCondition[]`).

- **Editor/export divergence.** Logic is implemented twice — once in `App.tsx` (Preview) and once in `utils/exportHtml.ts` (shipped runtime). A behavior that works in Preview is not guaranteed to work after export; `scripts/smoke-*.mjs` exist precisely because of this risk.
- **Quests are records, not state machines.** Only active/completed status + a flat objective list. Per the audit, `talk_to` and `skill_check` objectives aren't reliably evaluated at runtime, rewards aren't applied consistently, and there are no stages/branches. Confirmed by the messy objective data above.
- **Time is cosmetic.** A clock ticks and needs decay, but there is **no `day` counter persisted, no sleep, no schedules, no daily resets** — yet the game design requires "repeatable unlimited days." (`RuntimeGameState` has a `day` field, but save/load omits much of the runtime state.)
- **No first-class systems for the game's core fantasy.** Missing/'"needs redesign"' per the audit: rumors/gossip (the telephone quest), scene variants/anomaly layers (currently faked by duplicating whole scenes — hence the duplicate names), global story phases (Paleo→Glitching→Backrooms), gift history, visit counters/cooldowns, NPC schedules, bartering/economy, and a project-level event-rule engine ("when → if → do").
- **Repetitive ambient authoring.** Every clickable prop must be configured individually; no batch/recipe authoring for "40 silly clickables per room."

---

7. Proposed Clean JSON Schema for the Improved App

Design goals: **stable machine ids everywhere (never prose), one shared condition/action vocabulary, systems as first-class collections that objects reference rather than privately duplicate, quests as state machines, non-destructive scene variants, and a single runtime shared by editor and export.** Backwards-compatible via an importer modeled on Anzu's `ProjectCompatibility.normalize()`.

```
{
  "schemaVersion": 2,
  "id": "proj_uuid",
  "name": "Cavebot",
  "meta": { "author": "", "createdAt": "", "updatedAt": "", "startSceneId":
"scene_uuid" },

  // ---------- CONTENT ----------
  "scenes": [{
    "id": "scene_uuid",
    "name": "Wetlands (Sunset)",
    "slug": "wetlands_sunset",              // human-stable, unique, for readable ref-
erences
    "size": { "w": 800, "h": 600 },
    "background": { "color": "#000", "bgmAssetId": "asset_uuid" },
    "objects": [ /* SceneObject, see below */ ],
    "variants": [{                          // NON-DESTRUCTIVE anomaly/phase layers
      "id": "var_uuid",
      "name": "glitching",
      "activateWhen": "cond_group_ref",     // rule group id
      "priority": 10,
      "overrides": { "objectId": { "src": "asset_uuid", "hidden": false, "filters":
{} } },
      "addObjects": [], "removeObjectIds": []
    }]
  }],

  "uiMenus": [ /* same shape as Scene + { isOpenByDefault, blocksClicks, closeOnClick-
Outside } */ ],

  "assets": [{
    "id": "asset_uuid",
    "kind": "image|audio|video|script|font|hitbox",
    "name": "shoebill_idle.gif",
    "source": {                             // explicit, unambiguous resolution
      "mode": "linked|embedded|repo",
      "url": "https://i.pinimg.com/originals/0d/3b/9f/
0d3b9fc067eea3b425128821259cde04.gif", "dataRef": "blobstore_key", "repoPath": "as-
sets/…"
    },
    "dims": { "w": 0, "h": 0 },
    "tags": [], "category": "creatures", "audio": { "trimStart": 0, "trimEnd": 0,
"volume": 1 }
  }],

  "characters": [{
    "id": "char_uuid",
    "name": "Gilgrokmesh",
    "slug": "gilgrokmesh",
    "portraitAssetId": "asset_uuid",
    "factionIds": ["faction_uuid"],
    "relationshipTracks": [{                // MULTIPLE named tracks (trust, affec-
tion, respect…)
      "id": "track_uuid", "key": "trust", "label": "Trust",
      "min": -100, "max": 100, "default": 0,
      "thresholds": [{ "at": -30, "label": "Wary", "color": "#f97316" },
                     { "at": 70,  "label": "Bonded", "color": "#22c55e" }],
      "onThreshold": [{ "at": 70, "actions": ["action_ref"] }]   // threshold-
triggered events
    }],
    "giftPreferences": [{ "itemId": "item_uuid", "delta": 8, "trackKey": "affection",
"reaction": "…", "oncePerDay": true }],
```

```
    "boundaries": { "scritch": false },      // named permission/consent facts
    "relations": [{ "toCharId": "char_uuid", "kind": "sibling|spouse|rival|parent|
friend", "label": "", "mutual": true }],
    "schedule": [{ "days": ["*"], "from": 8, "to": 12, "sceneId": "scene_uuid" }]  //
time/day placement
  }],

  "factions": [{ "id": "faction_uuid", "name": "Hyenas", "role": "species",
                 "default": 0, "reputationLabel": "Standing", "allyId": null, "rival-
Id": null,
                 "consequences": [{ "when": "cond_group_ref", "do": ["ac-
tion_ref"] }] }],

  "companions": [{ "id": "comp_uuid", "characterId": "char_uuid", "assetId": "as-
set_uuid",
                   "followWhen": "cond_group_ref", "dialogueTreeId": "tree_uuid",
                   "interjections": [{ "text": "", "when": "cond_group_ref", "cool-
down": 3 }] }],

  "items": [{
    "id": "item_uuid", "name": "Bone Flute", "slug": "bone_flute",
    "description": "", "iconAssetId": "asset_uuid",
    "category": "quest|consumable|ingredient|tool|station|normal",
    "stackable": true, "maxStack": 99,       // quantities/stacks (currently missing)
    "use": { "consume": false, "message": "", "soundAssetId": null, "effects": ["ac-
tion_ref"] },
    "progress": { "key": "flute_repair", "steps": 3 }   // progress-track items (re-
pair/practice)
  }],

  "recipes": [{                              // ONE crafting model (flexible)
    "id": "recipe_uuid", "name": "", "stationItemId": null,
    "requirements": [{ "itemId": "item_uuid", "qty": 1, "consume": true, "role": "in-
gredient|tool" }],
    "skillGate": { "skillId": "skill_uuid", "min": 2 },
    "outcomes": ["action_ref"], "discoverable": true
  }],

  "dialogueTrees": [{
    "id": "tree_uuid", "name": "", "startNodeId": "node_uuid",
    "nodes": [{
      "id": "node_uuid", "speakerCharId": "char_uuid", "portrait": "right", "text":
"",
      "choices": [{
        "id": "choice_uuid", "text": "", "nextNodeId": "node_uuid|null",
        "showWhen": "cond_group_ref",        // condition GROUP (all/any/none), not a
single flag
        "actions": ["action_ref"]            // unified action list (give item, set
fact, rep delta, …)
      }]
    }]
  }],

  "quests": [{                               // STATE MACHINE, not a flat record
    "id": "quest_uuid", "name": "", "description": "", "autoStartWhen":
"cond_group_ref|null",
    "stages": [{
      "id": "stage_uuid", "name": "", "journal": "",
      "objectives": [{
        "id": "obj_uuid",
        "type": "talk_to|collect|reach_scene|skill_check|fact|custom",
        "target": { "kind": "character|item|scene|fact|skill", "id": "…" },   //
TYPED, never prose
```

```
        "count": 1, "optional": false
      }],
      "onEnter": ["action_ref"], "onComplete": ["action_ref"],
      "branches": [{ "when": "cond_group_ref", "goToStageId": "stage_uuid" }]
    }],
    "rewards": ["action_ref"]
  }],

  "maps": [{ "id": "map_uuid", "name": "", "backgroundAssetId": "asset_uuid",
             "nodes": [{ "id": "node_uuid", "name": "", "pos": {"x":0,"y":0},
                         "targetSceneId": "scene_uuid", "unlockWhen":
"cond_group_ref" }],
             "edges": [{ "fromNodeId": "", "toNodeId": "", "travelTime": 1 }] }],

  "lore": [{ "id": "lore_uuid", "title": "", "content": "", "type": "lore|journal|
quest_note",
             "category": "", "questId": null, "revealWhen": "cond_group_ref" }],

  // ---------- FACTS & RUMORS (replaces free-text flags) ----------
  "facts": [{ "id": "fact_uuid", "key": "shoebill_met", "label": "Met the shoebill",
              "type": "bool|number|enum", "default": false, "values": [] }],
  "rumors": [{ "id": "rumor_uuid", "subjectCharId": "char_uuid", "text": "", "accur-
acy": "true|false|distorted",
               "knownByCharIds": [], "learnWhen": "cond_group_ref" }],

  // ---------- SHARED LOGIC (the anti-fragmentation core) ----------
  "conditionGroups": [{                       // referenced everywhere by id
    "id": "cond_group_ref", "mode": "all|any|none",
    "conditions": [{
      "type": "fact|item|quest_stage|skill|need|relationship|time|day|scene|phase|vis-
it_count",
      "target": { "kind": "…", "id": "…", "trackKey": "trust" },
      "op": "is|is_not|gte|lte|gt|lt", "value": 0
    }],
    "children": ["cond_group_ref"]           // nestable
  }],

  "actions": [{                               // referenced by objects/choices/quests/
etc.
    "id": "action_ref",
    "type": "show_text|play_sound|play_animation|give_item|remove_item|exchange_item|
             set_fact|adjust_relationship|adjust_skill|adjust_need|adjust_reputation|
             start_quest|advance_quest|complete_quest|change_scene|set_scene_variant|
             set_story_phase|queue_event|open_ui",
    "params": {}
  }],

  "eventRules": [{                            // PROJECT-LEVEL "when → if → do" engine
    "id": "rule_uuid", "name": "Shoebill song unlock",
    "trigger": "on_enter_scene|on_next_visit|on_day_start|on_time|on_fact_change|
on_interact",
    "triggerParams": { "sceneId": "scene_uuid" },
    "when": "cond_group_ref",
    "do": ["action_ref"],
    "once": true, "cooldownDays": 0
  }],

  "scheduledEvents": [{ "id": "ev_uuid", "at": { "day": 3, "time": 9 }, "when":
"cond_group_ref", "do": ["action_ref"], "repeat": "none|daily" }],

  "storyPhases": [{ "id": "phase_uuid", "key": "paleo|glitching|backrooms", "order":
0,
                    "enterWhen": "cond_group_ref", "onEnter": ["action_ref"] }],
```

```
  // ---------- OBJECTS reference systems, don't duplicate them ----------
  //  SceneObject = { id, name, assetId, trans-
form{x,y,w,h,rotation,zIndex,opacity,flip},
  //                  render{blendMode,parallax,filters,animation}, cursor,
  //                  characterId?, isHitbox?, isText?, text{...}, ui{...},
  //                  showWhen: "cond_group_ref",
  //                  onClick: { responses: [{ when: "cond_group_ref", do: ["ac-
tion_ref"], once? }] },
  //                  triggerOnEnter?: "action_ref" }

  "settings": {
    "stage": { "w": 800, "h": 600, "grid": 32 },
    "time": { "mode": "action|realtime|manual", "startHour": 8, "hoursPerTick": 0.1,
"enableDays": true },
    "systems": { "needs": true, "skills": true, "relationships": true, "crafting":
true, "maps": true },
    "needs": [{ "id": "need_uuid", "key": "hunger", "label": "Hunger", "min": 0,
"max": 100, "default": 100, "decayPerTick": 1, "showInHud": true }],
    "skills": [{ "id": "skill_uuid", "key": "cooking", "label": "Cooking", "min": 0,
"max": 100, "default": 0, "showInHud": false }],
    "theme": { "preset": "retro", "primary": "#fbff00", "secondary": "#bc7f2a", "back-
ground": "#6b6bff", "font": "'Press Start 2P', monospace", "borderRadius": 4, "custom-
Css": "" },
    "dialogue": { "position": "below", "widthPct": 86, "textSizePx": 8, "portrait-
SizePx": 15, "typewriterSpeed": 0 },
    "hud": { "hideAllDefault": true, "overlayAssetId": null, "deviceFrame": null }
  },

  // ---------- RUNTIME SAVE STATE (must be COMPLETE) ----------
  "runtimeStateSchema": {
    "currentSceneId": "", "day": 1, "time": 8, "storyPhaseKey": "paleo",
    "facts": {}, "inventory": [{ "itemId": "", "qty": 1 }],
    "skills": {}, "needs": {}, "relationships": { "charId": { "trust": 0 } },
    "reputation": {}, "questProgress": { "questId": { "stageId": "", "objectives":
{} } },
    "sceneVariants": {}, "visitCounts": {}, "giftHistory": [], "firedRuleIds": [],
    "queuedEvents": [], "unlockedLore": [], "knownRumors": []
  }
}
```

7.1 What this schema fixes (mapped to §6)

- **Prose flags → `facts` with stable `key` + `id`;** dialogue/quests reference the id, so labels can change freely. Empty/duplicate identifiers become impossible.
- **One condition vocabulary (`conditionGroups`) and one action vocabulary (`actions`)** shared by objects, dialogue choices, quest stages, faction consequences, and the new `eventRules` — eliminating the `showIfFlag` vs `RuleCondition[]` and the triple `npcId` / `factionId` / `characterId` inconsistencies (all collapse into typed `target.{kind,id,trackKey}`).
- **Quests become state machines** (stages, typed objectives, onEnter/onComplete/branches) so `talk_to` / `skill_check` are first-class and rewards apply deterministically.
- **Scene `variants`** replace scene duplication for anomalies/phases (kills the `"meethyenaba"` duplicates); `storyPhases` make Paleo→Glitching→Backrooms first-class.
- **`eventRules` + `scheduledEvents` + `day` / `visitCounts` / `giftHistory`** give the shoebill courtship, telephone rumor chain, flute repair, NPC schedules, and daily resets a real home instead of flag soup.
- **Single crafting model, stackable items with quantities, explicit `asset.source`** remove the parallel-model and asset-resolution ambiguity.

- `runtimeStateSchema` **is exhaustive** so save/load and editor-Preview/export use one complete state object — closing the editor/export divergence gap, best enforced by shipping the Anzu-style **single shared engine** consumed by both.

---

8. Key Takeaways for the Rebuild

1. **Adopt the Anzu (abacus) architecture** — modular, toggleable, single-source engine shared by editor Preview and HTML export — while porting the main React app's much larger feature set onto it.
2. **Replace the flag system with typed** `facts` **+ shared** `conditionGroups` / `actions` **+ a project-level** `eventRules` **engine.** This is the highest-leverage change and unblocks most of the missing gameplay in one move.
3. **Promote to first-class systems** the things the cavebot design actually needs but currently fakes: multi-track relationships with gift/boundary/threshold events, quest state machines, scene variants + story phases, days/schedules/visit-counts, and rumors.
4. **Write a** `normalize()` **/migration importer** (modeled on `ProjectCompatibility.normalize`) that upgrades existing v1 exports like `7262026_stripped.json` — repairing the broken links, duplicate names, legacy `start-dialogue`, orphan trees, and empty flag targets found above — into schemaVersion 2.